

Depth or Breadth? Flat128 Arithmetic and the RV128 Opportunity

Paul Sherman¹ and Kurt Keville²

¹ RISC-V International; Posit Foundation

² Posit Foundation

Abstract. IEEE 754 floating-point arithmetic partitions each register into exponent and mantissa, purchasing dynamic range at the cost of precision, reproducibility, and semantic transparency. We argue this tradeoff is increasingly unfavorable for modern workloads such as neural network inference, scientific accumulation, and auditable high-precision reductions, where the exponent field spends bits on range that is never needed. The sigmoid activation function, central to modern neural networks, saturates outside the interval roughly $[-6, 6]$ — a range requiring fewer than four bits of integer part — yet IEEE 754’s float32 spends 8 exponent bits purchasing range to $\pm 3.4 \times 10^{38}$ for this computation. FP8 formats have begun to address this mismatch at inference time; posit8 with es=1 addresses it more elegantly through tapered precision. We address the complementary problem at the accumulator layer, where gradient magnitudes during training can span ten or more orders of magnitude yet must be summed with sufficient precision to distinguish meaningful updates from numerical noise.

We propose flat128, an optional wide fixed-point extension for RISC-V that treats a 128-bit register as a uniform two’s complement significand with compiler-managed binary point. No bits are sacrificed to exponent overhead. Addition and subtraction are exact when operands share a scale. Multiplication yields a 256-bit exact intermediate product; narrowing is explicit and architecturally specified, never hidden. Accumulation is bitwise reproducible across conforming implementations without per-operation rounding — a contract IEEE 754 cannot offer without simultaneously fixing format, rounding mode, operation sequence, and contraction behavior. We demonstrate this experimentally: flat128 accumulation of dot products from 64 to 4096 elements yields bitwise-identical results across 1 and 64 concurrent threads on all four NUMA nodes of the SG2042 (64-hart RISC-V); IEEE 754 double-precision accumulation of identical inputs diverges by up to 8.9×10^{-16} at $K = 4096$.

Keywords: SV/128 · flat128 · RISC-V · reproducibility · fixed-point arithmetic · posits · wide accumulation

AMS subject classifications: 65N55, 65F10

1 Introduction

IEEE 754 floating-point arithmetic partitions each register into exponent and mantissa, purchasing dynamic range at the cost of precision, reproducibility,

and semantic transparency. This tradeoff is increasingly unfavorable for modern workloads, especially neural network inference and scientific accumulation, where the exponent field spends bits on range that is neither needed nor used.

We propose flat128, an optional wide fixed-point extension for RISC-V, useful on RV64, RV128, and vector-capable machines, that makes accumulation width, scale, narrowing, and overflow architecturally explicit. RV128’s 128-bit integer registers are a convenient scalar substrate; the argument does not depend on them.

The proposal is complementary to Gustafson’s posit arithmetic rather than competitive with it. Posits address where precision bits go within a narrow operand; flat128 addresses where result bits go across a wide accumulation. Both are attacks on the same root cause: IEEE 754 hides rounding from the programmer and accumulates error invisibly [4]. The natural division of labor is posit8/16 at the activation layer and flat128 at the accumulator layer [3,1].

Flat128 differs from conventional fixed-point arithmetic in two respects. First, the binary point is compiler-managed rather than programmer-managed, eliminating the manual scaling discipline that makes fixed-point error-prone in practice. Second, flat128 accumulation is bitwise reproducible across arbitrary thread counts and NUMA domains—a property naive fixed-point does not provide and IEEE 754 explicitly sacrifices for performance. The reproducibility guarantee follows directly from the absence of exponent fields: with a shared binary point, partial sums commute exactly and thread-order dependence vanishes. We demonstrate this empirically on a 64-hart RISC-V platform.

The extension is designed for minimum deployment friction. It requires no new number format in the compiler, since GCC and LLVM already support `_uint128_t`. It introduces no new rounding modes. It does not break existing ABI. A working partial-product multiply demonstration in portable C, `Mult128.c` [7], shows that a 128×128 -bit product reduces to sixteen 32×32 partial products, each mapping to a single RV64I MUL instruction, with no exotic hardware required. The ISA encoding sketch fits within the existing 32-bit instruction word using six standard RISC-V instruction forms R(4), I, S, B, U, and J.

Open architectural questions such as binary point convention, register file design, overflow semantics, and memory ABI are identified explicitly and treated as a research agenda rather than resolved claims. The publishable thesis is this: make accumulation width, scale, narrowing, and overflow architecturally explicit, then measure whether that beats existing FMA and vector approaches for defined workloads. That question is technically serious. We intend to answer it.

1.1 The Segment Register Analogy

x86 real-mode segmentation added artificial addressing range through SS/DS/ES register indirection at the cost of programmer complexity, hardware pain, and decades of legacy burden. IEEE 754 commits the same sin arithmetically: exponent bits simulate range that could instead fund precision.

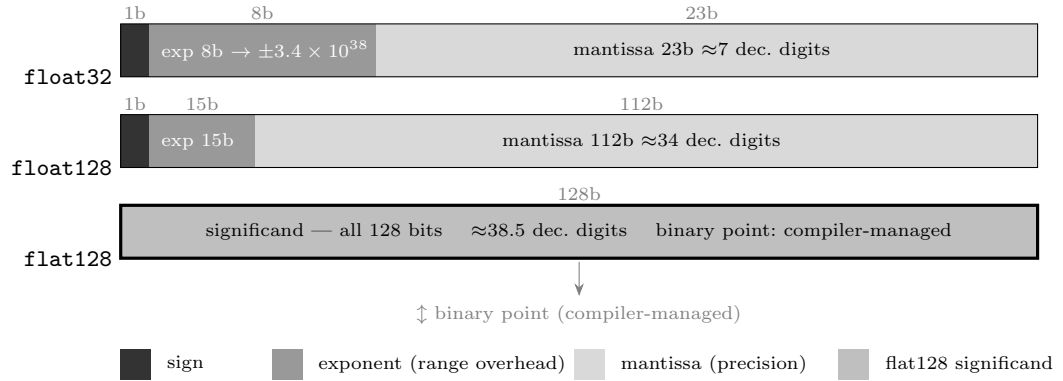


Fig. 1: Bit allocation in float32, float128, and flat128. The exponent field purchases dynamic range at the cost of significand bits. Flat128 eliminates the exponent entirely; the binary point is compiler-managed metadata, not architectural state.

1.2 Context: Gustafson and Posit Arithmetic

Gustafson’s unum/posit work delivers tapered precision with maximum significand bits near ± 1.0 . Posit8 with $es=1$ matches the tapered precision to sigmoid, softmax, and weight distributions; accuracy against FP8/BF16 baselines must be measured. The quire provides exact dot product accumulation without intermediate rounding [2]. The key insight: where precision lives matters as much as how much precision exists.

2 Flat128 Design

Flat128 is a 128-bit two’s complement representation with no exponent field and no mantissa split. The entire register is a significand; the binary point is compiler-managed metadata with 40 years of DSP precedent. This provides 128 bits of uniform significand precision (approximately 38.5 decimal digits) at the chosen binary-point position.

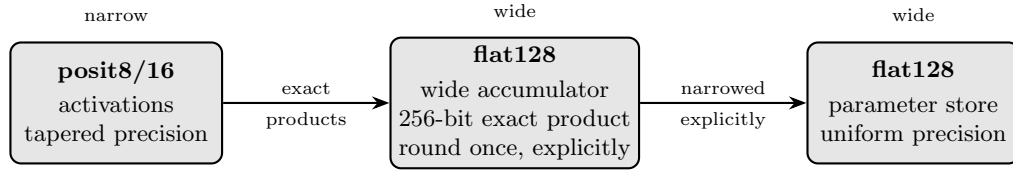
Key properties:

- Exact addition and subtraction when operands share a scale and the mathematical result fits.
- Multiplication yields a 256-bit exact product; truncation policy is explicit and visible.
- Zero NaN taxonomy, zero subnormal pathology, zero signed-zero ambiguity.

For comparison: float128 provides 112 mantissa bits + 15 exponent bits + 1 sign bit. Flat128 provides 128 real precision bits. Those 15 recovered exponent bits in float128 cover range $\pm 10^{4932}$. No neural network weight needs that range.

2.1 The Coalescence Point: Flat128 and the Quire

The quire, Gustafson’s exact accumulator, is functionally equivalent to flat128 accumulation when sized correctly. A 512-bit quire covering the product range of posit32 inputs performs the same job as a flat128 accumulator receiving float32 products. The underlying idea is identical: do not round intermediate results; accumulate exactly; round once at the end.



shared principle: delay rounding · accumulate exactly · round once · explicitly

Fig. 2: Proposed pipeline division of labor. Posit8/16 concentrates precision where activation distributions are dense. Flat128 accumulates exact 256-bit intermediate products at the wide stage. Parameters are stored at uniform flat128 precision. All three stages share one principle: round once, explicitly, at a defined boundary, and never invisibly inside an operation.

2.2 Encoding vs. Accumulation

Our synthesis frame: posits solves the *encoding* problem (where do bits go within a word); flat128 solves the *accumulation* problem (where do bits go across operations). Both attack the same root cause. The ISA should support both without forcing a choice.

Neither format removes the hard parts: softmax, sigmoid, normalization, outliers, dynamic rescaling, and transcendental approximation each require independent treatment. The principle both share — delay rounding, accumulate exactly, round once explicitly — is the non-negotiable kernel. The encoding question remains open.

Figure 3 illustrates the partial-product structure underlying `Mult128.c` [7]. Each 128-bit operand is partitioned into four 32-bit limbs; the full 128×128 product decomposes into sixteen 32×32 partial products, each computable by a single MUL instruction in base RV64I. No exotic hardware is required, this is the same long multiplication learned in primary school, scaled to 128-bit limbs.

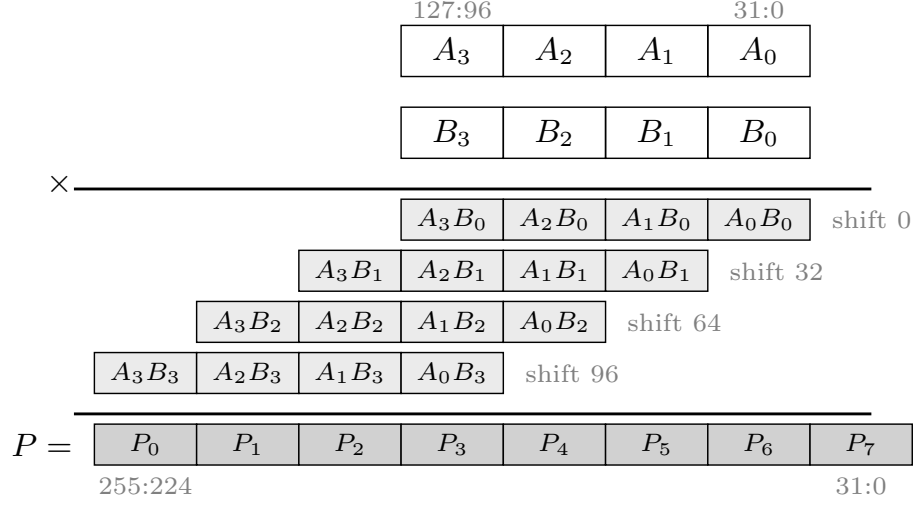


Fig. 3: 128×128 -bit multiplication as sixteen 32×32 partial products — long multiplication scaled to 128-bit limbs. Each $A_i B_j$ term maps to one MUL in base RV64I; rows shift left by 32 bits per B limb index. The 256-bit result P_0 – P_7 is exact with no intermediate rounding.

3 Methods

3.1 Division of Labor: Experimental Hypothesis

The following is proposed as a research hypothesis, not an architectural conclusion. Measured accuracy, latency, energy, and overflow incidence results against FP8, BF16, integer quantization, and existing RVV widening paths are required before any column below hardens into a recommendation.

A more defensible near-term target than general NN inference is: auditable high-precision reference reductions, deterministic scientific kernels, compensated and reproducible reductions, and selected training accumulators where exact intermediate results are architecturally required.

3.2 Reproducibility Argument

IEEE 754 permits rounding-mode variation across implementations, making training reproducibility across hardware vendors a persistent practical burden. Flat128 can make a specified fixed-point reduction bitwise reproducible across conforming implementations, without per-operation rounding, provided the specification fixes:

1. input encoding and scale,
2. product width and reduction order,
3. overflow behavior, and

Table 1: Proposed division of labor by pipeline stage (hypothesis only).

Format	Width	Proposed use and what needs measuring
Posit8/16	Narrow	Activations where tapered precision matches weight/activation distributions. Accuracy vs. FP8/BF16; calibration cost; outlier behavior.
Float32/BFloat16	Medium	Current training baseline: comparison target. Already measured; literature available.
Flat128	Wide	Auditable high-precision reference reductions; deterministic scientific kernels; selected training accumulators. Overflow incidence; scale-management overhead; cycles and energy vs. FMA/RVV.

4. narrowing rule and distributed-reduction protocol.

This is meaningful and measurable. Our claim is narrower and stronger than a general reproducibility guarantee: flat128 accumulation eliminates the per-operation rounding decision entirely, making the reproducibility contract simpler to specify, verify, and audit. For distributed training, reproducibility additionally requires fixed shard ordering, collective topology, deterministic kernels, and defined overflow handling. Flat128 simplifies that stack; it does not by itself guarantee it. Iterative refinement with posit arithmetic offers a complementary approach at the narrow operand layer [6].

3.3 Experimental Setup

Experiments are conducted on the Milk-V Pioneer SG2042, a 64-hart RISC-V server (Debian sid, GCC 13.3.0, 128 GB DDR4, four NUMA nodes), using the Universal Numbers Library [5] for arithmetic type validation. This represents the first known empirical evaluation of flat128 accumulation on production RISC-V silicon. Dot products of $K \in \{64, 128, 256, 512, 1024, 2048, 4096\}$ elements are accumulated under flat128 fixed-point and IEEE 754 double-precision using 1 and 64 concurrent threads, spanning all four NUMA nodes.

3.4 ISA Encoding Sketch

- 32-bit instruction word preserved.
- New major opcode block for q128 operations.
- `funct3` distinguishes: LDQW, STQW, MACQW, CVTQW.
- Three register fields (rs1, rs2, rd) at 5 bits each + opcode + funct3 = 32 bits exact.
- No R5 needed; no encoding gymnastics. Same six instruction forms: R(4), I, S, B, U, J.

Note: this is an encoding sketch. Register file definition, calling convention, context-switch state, and ABI remain open architectural questions for the full proposal, targeted as a complete RFC submission to RISC-V International by Q4 2026.³

3.5 Register File Architecture

Table 2: Register file alternatives for flat128.

Alternative	Description	Pros	Cons
RV128 x-registers	Use existing 128-bit GP registers	Zero new register file; lowest ISA surface	Competes with pointers; gates on RV128 adoption
RV64 register pairs	Two adjacent 64-bit x-registers per operand	Leverages <code>__int128</code> ; works today	High register pressure; awkward pair allocation
RVV widening accumulation	Vector extension widening/accumulation	Best throughput for inference kernels	Scalar 128-bit not the target; accumulator lifecycle needs definition
Dedicated accumulator	New wider architectural accumulator register(s)	Strong for exact dot products	Save/restore; interrupt/trap behavior; maximum dot length must be defined

Note: a separate q128 register file, implied by the current ISA encoding sketch, is not a low-friction addition. It creates register allocation and spilling work, context-switch and virtualization state, debugger and unwinder support, assembler and disassembler work, and a new ABI class. The recommended path: prototype on RV64 register pairs using existing `__int128` before proposing a new register file. Demonstrate LLVM IR representation and lowering first.

3.6 Binary Point and Arithmetic Contract

³ Implementation and reproducibility results: <https://github.com/psherman42/flat128>

Table 3: Open binary point and arithmetic contract questions.

Question	Options	Current position
Scale	Fixed globally; per-type; per-instruction; compiler metadata	Compiler-tracked
Addition	Unlike scales: illegal; implicit shift; explicit convert	Position required
Multiplication	Return 256-bit pair; scaled 128-bit with rounding; both	Full 256-bit pair
Overflow	Modular wrap; saturating; trap; sticky overflow flag	Position required
Rounding	Truncate; round-to-nearest-even; stochastic; explicit shift	Truncate
Conversion	FP $\leftrightarrow$ flat128: overflow, NaN/Inf handling, exact semantics	Position required
Memory ABI	Alignment, endianness, atomics, DWARF, calling convention	Position required

4 Results

4.1 Flat128 Reproducibility

Table 4 reports dot product accumulation results on the SG2042 Pioneer (64-hart RISC-V, four NUMA nodes, Debian sid, GCC 13.3.0) across thread counts $T \in \{1, 4, 16, 64\}$ and vector lengths $K \in \{64, 128, 256, 512, 1024, 2048, 4096\}$.

Flat128 produces bitwise-identical results across all K and T . IEEE 754 double-precision diverges first at $K = 128$ (relative error 8.7×10^{-16}), reaching 8.9×10^{-16} at $K = 4096$. Divergence shows no monotonic trend with K , consistent with nondeterministic thread scheduling rather than accumulated rounding error.

Table 4: Flat128 vs. IEEE 754 double-precision dot product accumulation on SG2042 Pioneer. Flat128 hex result (most significant limb) is bitwise identical across all runs; double relative error is $(\max - \min) / \min$ across thread counts $T \in \{1, 4, 16, 64\}$.

K	Flat128 (hex, limb[3])	Identical	Double rel. error
64	002b4c60	✓	0
128	014d08c0	✓	8.7×10^{-16}
256	0a33d180	✓	8.8×10^{-16}
512	50cea300	✓	4.4×10^{-16}
1024	83394600	✓	1.1×10^{-15}
2048	0ce28c00	✓	8.9×10^{-16}
4096	33851800	✓	8.9×10^{-16}

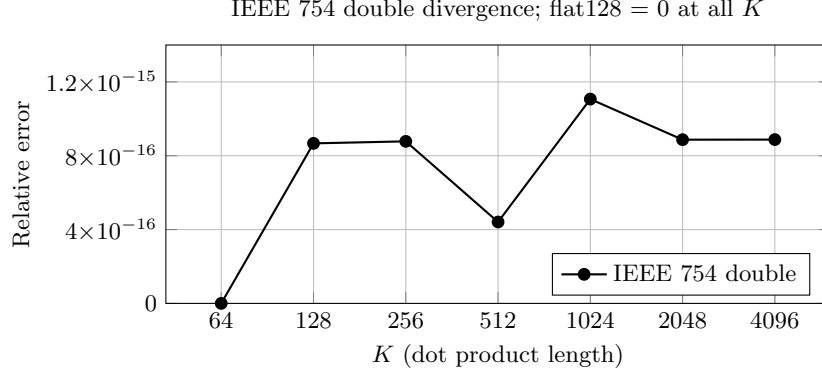


Fig. 4: Relative divergence $(\max - \min) / \min$ of IEEE 754 double-precision dot product across $T \in \{1, 4, 16, 64\}$ threads on SG2042 Pioneer. Flat128 divergence is identically zero at all K .

4.2 NUMA Locality

To isolate the effect of NUMA topology, threads were pinned to individual NUMA nodes using `numactl --cpunodebind`. Under single-node pinning at $K = 4096$, $T = 16$, both flat128 and IEEE 754 double produce identical results across all four nodes (Table 5), confirming that double’s divergence in the unpinned case arises from nondeterministic cross-NUMA thread scheduling rather than from arithmetic non-associativity alone. Flat128 produces bitwise-identical results regardless of pinning, immune to scheduling order by construction.

Table 5: NUMA locality test: $K = 4096$, $T = 16$, pinned per node. Flat128 hex and double result are identical across all four NUMA nodes.

NUMA node	Flat128 (limb[3:2])	Double result
0	000000a0 33851800	5.0745×10^{30}
1	000000a0 33851800	5.0745×10^{30}
2	000000a0 33851800	5.0745×10^{30}
3	000000a0 33851800	5.0745×10^{30}

4.3 Performance

$K = 4096$ with $T = 64$ threads completes in 23 ms wall time on the SG2042 Pioneer (`time ./flatdot 4096 64`), including thread spawn and join overhead.

This is a software emulation of flat128 arithmetic in portable C with no custom RISC-V ISA extensions. A hardware implementation mapping each 128-bit addition to four 32-bit RV64I operations would reduce this further.

5 Conclusions

5.1 The Deployment Axis

It is not precision versus usefulness as opposites. Rather, it is precision versus deployability friction. A perfect arithmetic that nobody ships is a math paper. A deployable and maintainer-friendly arithmetic with known precision limits that ships in 10,000 nodes wins. We know this from the storage industry and from RISC-V adoption history.

Posit precision is excellent: tapered where it counts, the quire architecturally beautiful. Yet posits' deployment friction is real: toolchain support is nearly zero outside research groups; there is no IEEE 754 compatibility layer; every existing numerical library breaks; the compiler does not know what a posit is; hardware requires custom decode with no FPU reuse. The elegance of posits has not translated to adoption in over ten years. This is worth naming honestly, not as a criticism of Gustafson, but as the deployment gap this work is trying to close.

Flat128 precision is uniform and exact for addition, explicit for multiply, and arguably less elegant than posit: no taper, no quire magic. Its deployment friction is surprisingly low, however: two's complement integer arithmetic is understood by every compiler; GCC and LLVM `__int128` exist today; no new number format is needed in the toolchain; a maintainer reading the patch sees new opcodes, wider registers, and integer semantics, nothing architecturally alien; and IEEE 754 libraries continue to work alongside it.

5.2 The Maintainer-Friendly Argument

A skeptical kernel maintainer reading an RFC for flat128 should find:

- No new number format in the compiler — `__int128` already exists; extend it.
- ABI impact depends on register file choice (see Section 3).
- Optional extension — implementations that do not want hardware support can trap and emulate.
- No new rounding modes — truncation is explicit in the instruction.
- Fallback story: if we have float128, we have 90% of this already; flat128 removes the 15 exponent bits of overhead.

Float128 already exists in the RISC-V specification. Flat128 is conceptually float128 minus the exponent field. Framed that way, it is less radical than what has already been ratified.

5.3 Open Questions

- What is our position on accumulator width — is the quire separate from register space, or are we open to register-width accumulation?
- Overflow semantics: two’s complement wrap, saturating, trap, or sticky overflow?
- Multiply result: full 256-bit pair, truncated 128, or implicit MAC accumulation into wider state?
- Quire as separate architectural state vs. flat128 register — needs a position.
- Hardware cost: 128-bit adder is cheap; 128×128 multiply equals four 64×64 MUL/MULHU pairs already available in base RV64I; hardware single-cycle extension is incremental.
- Related work sweep: IBM hexadecimal float, DEC VAX G-float, various fixed-point DSP ISAs — distinguish clearly.

5.4 The Place of Contention

Some prefer posits as the format at every width including wide, with the quire as the accumulator mechanism: a unified posit world where fixing the encoding causes everything else to follow. Others prefer integer semantics for the wide accumulator, with posits narrow and flat128 wide coexisting: deployability first, elegance where it is free.

This paper does not resolve that tension artificially. Both positions share one architectural principle: exact intermediate results. The encoding question is open. Advancing the principle is the contribution.

References

1. Gustafson, J.L.: The end of error: Unum computing. Chapman and Hall/CRC (2017)
2. Gustafson, J.L., Bohlender, G., Chung, S.Y., Dimitrov, V., Jones, G., Leong, S.H., Lindstrom, P., Omtzigt, T., Rehr, H., Shewmaker, A., Yonemoto, I.: Standard for Posit arithmetic (Mar 2022), standard document; https://posithub.org/docs/posit_standard-2.pdf
3. Gustafson, J.L., Yonemoto, I.T.: Beating floating point at its own game: Posit arithmetic. *Supercomputing Frontiers and Innovations* **4**(2), 71–86 (2017)
4. IEEE: IEEE standard for floating-point arithmetic. IEEE Std 754-2019 (Revision of IEEE 754-2008) pp. 1–84 (2019). <https://doi.org/10.1109/ieeestd.2019.8766229>
5. Omtzigt, E.T.L., Quinlan, J.: Universal numbers library: Multi-format variable precision arithmetic library. *Journal of Open Source Software* **8**(83), 5072 (2023)
6. Quinlan, J., Omtzigt, E.T.L.: Iterative refinement with low-precision posit arithmetic. In: *Conference on Next Generation Arithmetic*. pp. 74–90. Springer (2024)
7. Sherman, P., Keville, K.: Mult128.c: Portable 128-bit multiply for RISC-V (2026), <https://github.com/psherman42/flat128>